

ZOMATHON — CART SUPER ADD-ON (CSAO) RECOMMENDATION SYSTEM USING AI

Team Name: Lokesh Hackathon

Team Leader: Kanta Lokeswararao

Team Members:

- * Snehal Das

- * Pavan Kumar

4M
Total revenue

1.76K
Aov

ZOMATHON CART BEHAVIOUR ANALYSIS

6.02
Avg items per order

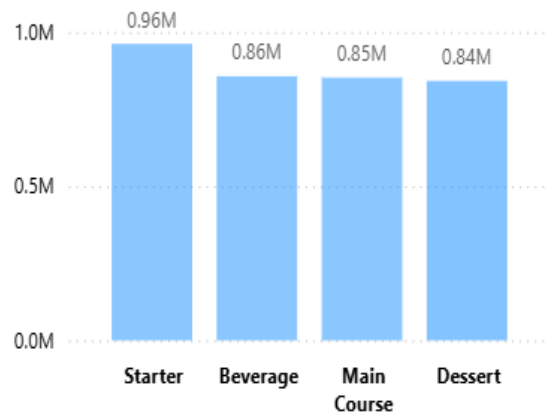
2K
total orders

city

Bangalore

Chennai

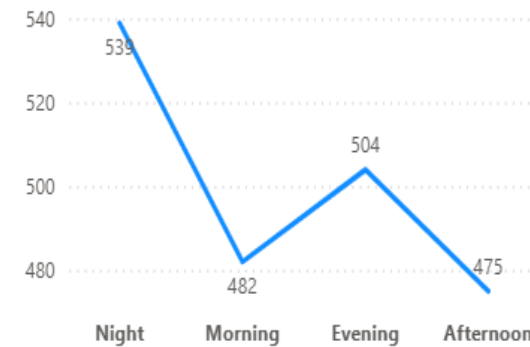
Revenue by Category



Category Contribution to Basket Size



Peak ordering Time For Context- Aware Recommendations



time_of_d...

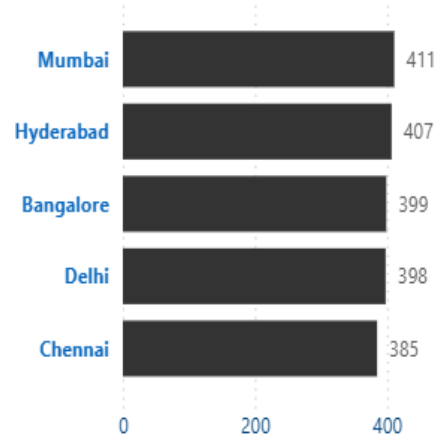
☐ Afterno.

☐ Evening

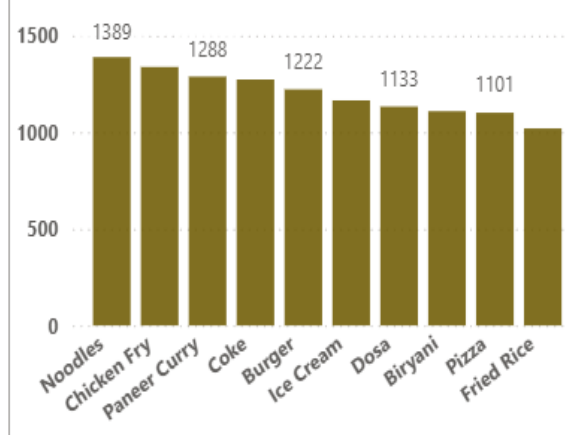
☐ Morning

☐ Night

Orders by City



Most Popular Items



Key Insights

- Starters generate the highest revenue among all categories.
- Average basket size is driven mainly by Starter and Main Course items.
- Night time shows the highest ordering activity.
- Mumbai leads in total orders across cities.
- Noodles and Chicken Fry are the most popular items.

Recommendation Strategy

Our recommendation strategy focuses on increasing Average Order Value (AOV) and improving customer experience through intelligent add-on suggestions.

Key Strategy Components:

Basket-Based Recommendations

Suggest complementary items based on items already present in the cart.

Context-Aware Suggestions

Recommendation's change based on time of day, location, and ordering patterns.

Popular Combination Mining

Identify frequently purchased item combinations using historical order data.

Personalized Recommendations

Adapt suggestions based on user preferences and past behaviour.

Real-Time Dynamic Updates

Continuously update recommendations as users add or remove items from the cart.

This strategy ensures higher conversions, better engagement, and optimized cart value.

Business Impact

Our proposed CSAO system delivers measurable business value for food delivery platforms:

- * Increase in Average Order Value (AOV) through smart add-on suggestions
- * Higher revenue per customer without increasing acquisition cost
- * Improved customer satisfaction with personalized recommendations
- * Better restaurant visibility through optimized item promotion
- * Reduced order cancellations using delivery risk prediction
- * Enhanced operational efficiency through delivery partner optimization

Overall, the system drives revenue growth, operational efficiency, and customer retention.

Real-World Delivery Challenges in Residential Communities

Food delivery platforms often face operational challenges when delivering orders to apartments and gated residential communities. These real-world issues impact delivery efficiency, customer satisfaction, and partner productivity.

Key Problems Identified

1. Customer Unavailability at Delivery Time

Customers may place orders and become unavailable due to travel, meetings, or other reasons. This leads to failed delivery attempts, delays, and order cancellations.

2. Entry Restrictions in Apartments and Gated Communities

Security verification requirements at residential complexes often delay delivery partners, especially when prior entry approval is not provided by the customer.

3. Lack of Pre-Delivery Confirmation Mechanism

Delivery partners frequently start deliveries without knowing whether the customer will be available at the destination, increasing uncertainty and inefficiency.

4. Communication Gaps Between Customer, Security, and Delivery Partner

Manual coordination between residents, security guards, and delivery partners can cause confusion, waiting time, and missed deliveries.

5. Delivery Partner Time Loss and Productivity Impact

Waiting at gates, repeated customer calls, and failed attempts reduce delivery partner efficiency and increase operational costs.

6. Increased Risk of Order Delays and Cancellations

Unavailability and entry barriers directly contribute to late deliveries, customer dissatisfaction, and revenue loss for platforms.

AI-Powered Delivery Optimization & Verification System

Automatic Apartment Entry Enable + Prediction + security Features

To support deliveries in apartments and gated residential communities, the system intelligently detects apartment-type addresses during order placement and automatically provides an entry enable option to the customer.

When a user enters or selects an address identified as an apartment or society, the app displays a prompt:

“Your address appears to be an apartment or gated community. Would you like to enable security entry support for faster delivery?”

Key Process:

* The system detects apartment-type addresses using location intelligence and address patterns

- * Customers can enable “Entry Support” during order placement
- * Once enabled, a secure verification token (QR or OTP) is generated for the delivery
- * Delivery partners present the token at the apartment gate for verification
- * Customers receive notifications before delivery arrival for confirmation if required

Customer Availability Prediction

The system sends a smart confirmation notification to the customer before the delivery partner starts.

Customer interaction signals such as response time, acknowledgment, or no response are used by AI to predict availability at delivery time.

If availability risk is detected, the system alerts the delivery partner and suggests optimized delivery timing.

This automated detection reduces manual effort and improves delivery efficiency in residential communities.

Benefits:

- * Faster entry approval at apartment gates
- * Reduced delivery delays
- * Improved delivery partner efficiency
- * Higher delivery success rate
- * Better customer experience and convenience

Proposed AI System Features

This AI-powered CSAO system is designed to increase Average Order Value (AOV) and enhance customer experience through intelligent, context-aware recommendations in food delivery platforms.

Key Features:

Add-On Recommendation Engine

Suggests complementary items based on cart composition and user behaviour patterns.

LLM-Based Food Intelligence

Provides taste profiles, nutrition details, dish descriptions, and personalized explanations for recommendations.

Personalized Suggestions

Adapts recommendations based on user preferences, ordering history, and spending patterns.

Context Awareness Engine

Considers time of day, location, and restaurant characteristics for dynamic recommendations.

Smart Restaurant Recommendation

Identifies the best restaurant for a selected dish using ratings, popularity, and delivery time.

Delivery Risk Prediction

Predicts potential delays or cancellations due to distance and demand and suggests optimized alternatives.

Delivery Partner Optimization

Assigns efficient delivery partners and routes to improve delivery success rates.

Dynamic Sequential Recommendations

Continuously updates suggestions as items are added or removed from the cart.

This integrated AI approach improves customer engagement, operational efficiency, and revenue growth.

Customer Availability Prediction

Predicts the likelihood of customer presence at delivery time using behavioural signals, response patterns, and historical delivery interactions.

Intelligent Address Classification

Automatically detects apartment or complex-type delivery locations using address intelligence and location data.

Security Verification Intelligence

Supports delivery partner identity validation and entry verification mechanisms using AI-assisted authentication methods.

AI System Workflow (Architecture)

User Input



Data Processing Layer



Intelligent Address Classification



AI Recommendation Engine



LLM Intelligence Layer



Restaurant Optimization Module



Delivery Risk & Customer Availability Prediction



Automatic Entry Notification & security Verification



Delivery Partner Notification & Routing



Final Recommendations to User

Our proposed system follows a multi-stage AI workflow to generate intelligent recommendations and optimize delivery efficiency.

Step 1: User Input

Customer selects items or searches for a dish (e.g., Biryani).

Step 2: Data Processing Layer

The system analyses cart composition, user preferences, location, time of day, and restaurant data.

Step 3 : Intelligent Address Classification

AI identifies whether the delivery location is an apartment, gated community, or independent house using address intelligence.

Step 4: Recommendation Engine

AI models identify the most relevant add-on items based on behavioural patterns and contextual factors.

Step 5: LLM Intelligence Layer

A Large Language Model generates explanations, food pairing suggestions, taste profiles, and nutrition insights.

Step 6: Restaurant Optimization Module

The system recommends the best restaurant based on ratings, distance, and delivery time.

Step 7: Delivery Risk & Customer Availability Prediction

AI predicts potential delivery delays, cancellation risks, and customer availability probability using logistics and behavioural signals.

Step 8: Automatic Entry Notification , security Verification & Delivery Partner Routing

The nearest delivery partner receives optimized routing instructions along with security verification support for apartment entry if required.

Step 9: Final Output to User

Customers receive personalized recommendations, explanations, estimated delivery time, and delivery tracking updates.

Future Enhancements

Voice-based food ordering using AI assistants

Health-aware recommendations based on dietary preferences.

An integrated SOS feature enables delivery partners and customers to quickly request emergency assistance during unsafe situations. The system can share live location, alert nearby support teams, and notify emergency contacts to ensure safety during deliveries. AI-based risk detection for unsafe delivery zones

```
# -----
```

```
# Add-On Recommendation Engine
```

```
# -----
```

```
def recommend_addons(cart_items, time_of_day):
```

```
    addons = {
```

```
        "Biryani": ["Raita", "Soft Drink", "Chicken 65"],
```

```
        "Pizza": ["Garlic Bread", "Coke", "Brownie"],
```

```
        "Burger": ["Fries", "Pepsi", "Nuggets"]
```

```
    }
```

```
    recommendations = []
```

```
    for item in cart_items:
```

```
        if item in addons:
```

```
recommendations.extend(addons[item])
```

```
# Context awareness
```

```
if time_of_day == "Night":
```

```
    recommendations.append("Ice Cream")
```

```
return list(set(recommendations))
```

```
# -----
```

```
# Customer Availability Prediction
```

```
# -----
```

```
def predict_customer_availability(response_received, response_time_minutes):
```

```
    if response_received and response_time_minutes <= 3:
```

```
        return "Customer Likely Available"
```

```
elif not response_received:
```

```
    return "Availability Uncertain — Send Reminder"
```

```
else:
```

```
    return "Customer May Be Unavailable"
```

```
# -----
```

```
# Apartment Address Detection
```

```
# -----
```

```
def detect_apartment(address):
```

```
    keywords = ["Apartment", "Apt", "Block", "Tower", "Gated", "Society"]
```

```
    for word in keywords:
```

```
        if word.lower() in address.lower():
```

```
            return True
```

```
return False
```

```
# -----
```

```
# Delivery Decision Engine
```

```
# -----
```

```
def delivery_decision(address, response_received, response_time):
```

```
    is_apartment = detect_apartment(address)
```

```
    availability = predict_customer_availability(response_received, response_time)
```

```
    if is_apartment:
```

```
        entry_message = "Enable Security Entry Pass (OTP/QR)"
```

```
    else:
```

```
        entry_message = "Normal Delivery"
```

```
if availability == "Customer Likely Available":
```

```
    delivery_action = "Start Delivery"
```

```
else:
```

```
    delivery_action = "Wait / Notify Delivery Partner"
```

```
return {
```

```
    "Address Type": "Apartment" if is_apartment else "Regular Location",
```

```
    "Entry Instruction": entry_message,
```

```
    "Availability Status": availability,
```

```
    "Delivery Action": delivery_action
```

```
}
```

```
# -----
```

```
# Example Scenario
```

```
# -----
```

```
cart = ["Biryani"]
```

```
time = "Night"
```

```
print("Recommended Add-ons:", recommend_addons(cart, time))
```

```
address = "Flat 302, Green Valley Apartment, Hyderabad"
```

```
result = delivery_decision(
```

```
    address=address,
```

```
    response_received=True,
```

```
    response_time=2
```

```
)
```

```
print("Delivery Decision:", result)
```

Dataset Link :

<https://docs.google.com/spreadsheets/d/1XCFXp1BJHxBLM9iijeT2biSjhnHWbEPH/edit?usp=drivesdk&ouid=108898113146942035465&rtpof=true&sd=true>

Thank you for the opportunity to present our solution. We believe this AI-powered system can significantly improve customer experience, delivery efficiency, and business growth in food delivery platforms.

We would be happy to answer any questions.